

MaixPy / MaixCAM 触摸屏使用方法

一、简介

对于 MaixCAM 自带了一个触摸屏，写应用时配合触摸屏可以实现很多有趣应用，我们可以通过 API 读取到触摸屏的点按操作。

二、MaixPy 读取触摸

MaixPy 提供了一个简单的

`maix.touchscreen.TouchScreen` 类来读取，举例：

```
1  from maix import touchscreen, appCopy
2  time
3
4  ts = touchscreen.TouchScreen()
5
6  pressed_already = False
7  last_x = 0
8  last_y = 0
9  last_pressed = False
10 while not app.need_exit():
11     x, y, pressed = ts.read()
12     if x != last_x or y != last_y or
13     pressed != last_pressed:
14         print(x, y, pressed)
15         last_x = x
16         last_y = y
17         last_pressed = pressed
18     if pressed:
19
```

```

19         pressed_already = True
20     else:
21         if pressed_already:
22             print(f"clicked, x: {x},
y: {y}")
                pressed_already = False
                time.sleep_ms(1) # sleep some
time to free some CPU usage

```

三、配合屏幕实现交互

配合屏幕可以做出一些用户交互的内容，更多可以看 [MaixPy/examples/vision/touchscreen](#) 目录下例程。

如前面的文章介绍的，我们要往屏幕显示内容，一般是得到一个 `maix.image.Image` 对象，然后调用

`disp.show(img)` 来显示这张图像。

实现一个按钮的最原始和简单的方法就是在这个图像上画一个按钮，然后判断用户触摸到这个区域就算是触发了按下事件，注意图像的大小要和屏幕的大小保持一致：

```

1  from maix import touchscreen, app,
2  time, display, image
3
4  ts = touchscreen.TouchScreen()
5  disp = display.Display()
6
7  img = image.Image(disp.width(),
8  disp.height())
9
10 # draw exit button
11 exit_label = "< Exit"
12 size = image.string_size(exit_label)
13 exit_btn_pos = [0, 0, 8*2 +
14 size.width(), 12 * 2 +
15 size.height()]
16 img.draw_string(8, 12, exit_label,
17 image.COLOR_WHITE)
18 img.draw_rect(exit_btn_pos[0],

```

```

19 exit_btn_pos[1], exit_btn_pos[2],
20 exit_btn_pos[3], image.COLOR_WHITE,
21 2)
22
23 def is_in_button(x, y, btn_pos):
    return x > btn_pos[0] and x <
    btn_pos[0] + btn_pos[2] and y >
    btn_pos[1] and y < btn_pos[1] +
    btn_pos[3]

    while not app.need_exit():
        x, y, pressed = ts.read()
        if is_in_button(x, y,
            exit_btn_pos):
            app.set_exit_flag(True)
            img.draw_circle(x, y, 1,
                image.Color.from_rgb(255, 255, 255),
                2)
            disp.show(img)

```

四、屏幕和图像大小不一样时如何处理

上面的例子可以看到 `img` 大小和屏幕大小一样，如果你的 `img` 和屏幕大小不一样怎么办（比如上面使用 `img = image.Image(240, 240)`，比如屏幕是 `640x480`，图像是 `240x240`，`disp.show(img)` 的默认行为是 `image.Fit.FIT_CONTAIN`，即把图片放大到 `480x480` 然后边上填充黑色，如果你在 `240x240` 的图上画了按钮，比如坐标 `(0, 0, 60, 40)`，那么按钮也会自动被放大，所以触摸判断的坐标就不能用 `(0, 0, 60, 40)`，需要用 `((640 - 480) / 2, 0, 480/240*60, 480/240*40)`，即 `(80, 0, 120, 80)`。

这里为了方便缩放图像时，快速计算源图像的点或者矩形框在缩放后的目标图像的位置和大小，提供了

`image.resize_map_pos` 函数来进行此计算过程。

```
from maix import touchscreen, app,
time, display, image

ts = touchscreen.TouchScreen()
disp = display.Display()

img = image.Image(240, 240)
img.draw_rect(0, 0, img.width(),
img.height(), image.COLOR_WHITE)

# draw exit button
exit_label = "< Exit"
size = image.string_size(exit_label)
exit_btn_pos = [0, 0, 8*2 +
size.width(), 12 * 2 +
size.height()]
img.draw_string(8, 12, exit_label,
image.COLOR_WHITE)
img.draw_rect(exit_btn_pos[0],
exit_btn_pos[1], exit_btn_pos[2],
exit_btn_pos[3], image.COLOR_WHITE,
2)
# 图像按键坐标映射到屏幕上的坐标
exit_btn_disp_pos =
image.resize_map_pos(img.width(),
img.height(), disp.width(),
disp.height(),
image.Fit.FIT_CONTAIN,
exit_btn_pos[0], exit_btn_pos[1],
exit_btn_pos[2], exit_btn_pos[3])

def is_in_button(x, y, btn_pos):
    return x > btn_pos[0] and x <
btn_pos[0] + btn_pos[2] and y >
btn_pos[1] and y < btn_pos[1] +
btn_pos[3]

while not app.need_exit():
    x, y, pressed = ts.read()
    if is_in_button(x, y,
```

```
exit_btn_disp_pos):
    app.set_exit_flag(True)
    # 屏幕的坐标映射回图像上对应的坐标，
    然后在图像上画点
    x, y =
image.resize_map_pos_reverse(img.width
img.height(), disp.width(),
disp.height(),
image.Fit.FIT_CONTAIN, x, y)
    img.draw_circle(x, y, 1,
image.Color.from_rgb(255, 255, 255),
2)
    disp.show(img,
fit=image.Fit.FIT_CONTAIN)
```